iOS-App mit PS4-Controller – Konzept

Nov 5, 2026 · @Lukas

Überblick

Die App ist eine reine iOS-App in Swift. Sie liest einen PS4-Controller (DualShock 4) aus und schickt daraus Fahrbefehle per Bluetooth Low Energy an den Pico 2w . Gleichzeitig zeigt sie alle Messwerte an, die der Pico 2w zurücksendet.

- **Eingabe:** PS4-Controller, per Bluetooth mit dem iPhone gekoppelt (normal in den iOS-Einstellungen). Ausgelesen über das Apple-Framework **GameController** (`GCController`, `GCDualShockGamepad`).
- **Verbindung zum Roboter: BLE** über **CoreBluetooth**, Nordic UART Service. Der Roboter taucht nicht in den iOS-Einstellungen auf, die App verbindet sich selbst.
- **Protokoll:** Die vorhandene Schnittstelle aus `docs/BLE_APP_SCHNITTSTELLE.md` (Branch `joystick`). Daten kommen als JSON-Zeilen, Befehle gehen als `$`-Textzeilen.
- **Firmware-Basis:** Branch `app` (abgeleitet von `joystick`). Die Linienfolge liegt auf `linienfolge` und wird später zusammengeführt.

Das Dokument legt fest, was die App kann und welche Daten fließen. Programmiert wird erst danach.

Daten vom Pico 2w (Ausgänge)

Nach dem Verbinden schaltet die App mit `$FMT,JSON` auf JSON um. Danach schickt der Pico 2w fünfmal pro Sekunde eine `status`-Zeile und bei Bedarf Ereignisse. Jede Zeile hat das Feld `"t"` (Typ).

status – alle 200 ms

```
{"t":"status","ms":123456,"mode":"joystick","line":
[0,1,0],"dist":0.153,"pwm":[45.0,45.0],"speed":[0.210,0.205],"pulses":
[412,409],"err":1.0,"trim":2.5,"maxSpeed":70}
```

Feld	Einheit	Bedeutung
`ms`	ms	Zeit seit Start des Pico 2w
`mode`	–	`idle`, `joystick`, `keys` (später zusätzlich `line`)

Feld	Einheit	Bedeutung
`line`	0/1 × 3	Spurfolgesensor links, Mitte, rechts; 1 = schwarze Linie
`dist`	m oder `null`	Abstand zum Hindernis vorne; `null` = kein Echo (frei > 4 m oder < 3 cm)
`pwm`	%	Sollwert links/rechts, −100 … 100 (negativ = rückwärts)
`speed`	m/s	Gemessene Radgeschwindigkeit links/rechts (immer ≥ 0)
`pulses`	Pulse	Lichtschranken-Pulse seit letztem Fahrbefehl (≈ 10 mm je Puls)
`err`	Pulse	Gleichlauf-Fehler (> 0: links voraus)
`trim`	%	Gelernte Motor-Korrektur
`maxSpeed`	%	Höchsttempo des Joysticks

Ereignisse

Typ	Beispiel	Bedeutung	Reaktion der App
`obstacle`	`{"t":"obstacle","dist":0.071,"brakeAt":0.074,"speed":0.16}`	Wegen Hindernis gebremst	Warnung + Controller vibriert
`timeout`	`{"t":"timeout"}`	300 ms keine Fahrbefehle → Pico hat angehalten	Hinweis „Verbindung gestört“
`msg`	`{"t":"msg","text":"…"}`	Freitext	Im Protokoll-Log
`info`	`{"t":"info","name":"Auto-Roboter","proto":1,…}`	Antwort auf `$INFO`	Version prüfen
`speed`	`{"t":"speed","max":70}`	Antwort auf `$SPEED`	Tempo-Anzeige aktualisieren
`fmt`, `pong`, `error`	–	Antworten auf `$FMT`, `$PING`, unbekannte Befehle	Log

Unbekannte Typen und Felder ignoriert die App, damit spätere Firmware mehr senden kann.

Controller-Belegung

R2 gibt Gas, L1/R1 wechseln den Modus, X startet, Kreis schaltet aus, Dreieck ist Not-Halt. Zum Lenken und Rückwärtsfahren fehlen in der Vorgabe noch Tasten, dafür gibt es unten Vorschläge.

Taste	Funktion	Herkunft
R2 (analog)	Geschwindigkeit vorwärts, 0–100 %	Vorgabe
L1 / R1	Modus zurück / weiter (Manuell ↔ Autonom)	Vorgabe
X	Start: autonomes Fahren beginnen; auch Neustart nach Aus oder Not-Halt	Vorgabe
Kreis	Aus: Motoren abschalten, Roboter rollt aus	Vorgabe
Dreieck	Not-Halt: sofort aktiv bremsen, alles gesperrt bis X	Vorgabe
Linker Stick (links/rechts)	Lenken im manuellen Modus	Vorschlag, nötig
L2 (analog)	Rückwärts, 0–100 %	Vorschlag, nötig
Quadrat	Feinmodus an/aus: Höchsttempo auf 30 % für genaues Rangieren	Vorschlag
Steuerkreuz ↑ / ↓	tempo im Autonomen fahren	Vorschlag
Steuerkreuz ← / →	Autonom: Lenkstärke der Linienfolge	Vorschlag
Touchpad wischen	App-Ansicht wechseln: Manuell / Autonom / Log	Vorschlag
Touchpad klicken	Gelernten Motor-Trim zurücksetzen (nur im Stand)	Vorschlag
Options	Einstellungen in der App öffnen	Vorschlag
L3 / R3	frei	–
PS-Taste	vom System belegt, nicht nutzbar	–

Rückmeldung am Controller (Vorschlag)

- **Lichtleiste** zeigt den Zustand (iOS kann die Farbe des DualShock 4 setzen): grün = Manuell bereit, blau = Autonom fährt, gelb = Hindernis/gedrosselt, rot = Not-Halt, weiß = keine Verbindung zum Roboter.
- **Vibration** bei Hindernis-Stopp (kurz) und Not-Halt (lang).
- **Akkustand** des Controllers in der App anzeigen.

Not-Halt und Aus

Not-Halt schließt die Motoren über den L298N kurz und bremst aktiv, Aus schaltet nur ab und lässt ausrollen. Beides ist in `MotorDriver` schon so umgesetzt (`brake()` bzw. `stop()`).

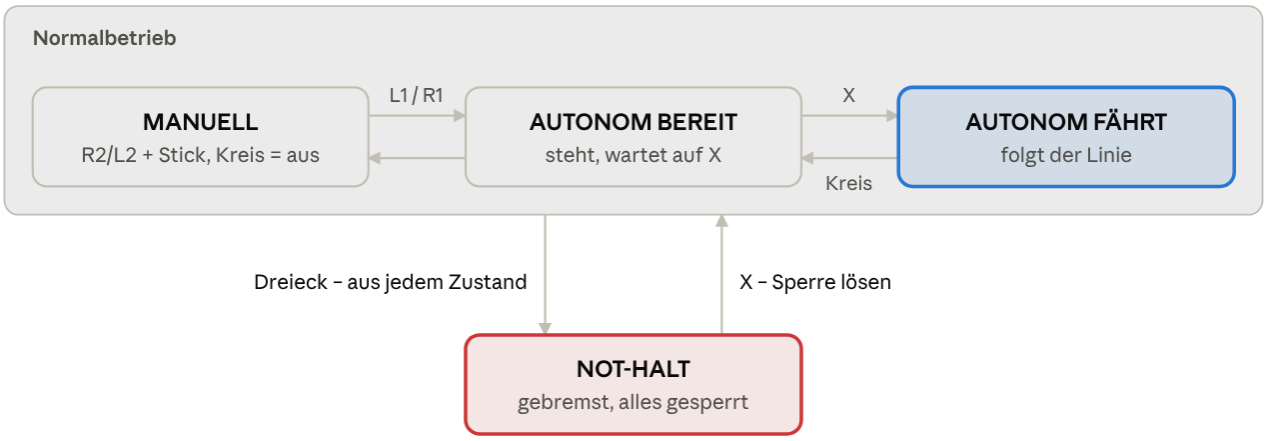
	Aus (Kreis)	Not-Halt (Dreieck)
Motortreiber	ENA/ENB = 0, IN1–IN4 = LOW	ENA/ENB = 100 %, IN1 = IN2 = HIGH, IN3 = IN4 = HIGH
Was am Motor passiert	Strom weg, Motor dreht frei	Motorklemmen kurzgeschlossen, der Motor bremst sich selbst
Wirkung	Rollt sanft aus	Steht nach sehr kurzem Weg
Bisheriger Befehl	`$STOP`	`$BRAKE`
Danach	Sofort wieder fahrbereit (R2 / X)	**Gesperrt:** kein Fahrbefehl wird angenommen, bis X gedrückt wird
Vorrang	normal	höchster: gilt in jedem Modus, auch während der Linienfolge

Die Sperre nach dem Not-Halt gibt es in der Firmware noch nicht. Bisher bremst `$BRAKE` nur einmal, ein neuer Fahrbefehl fährt sofort wieder los. Dafür ist der neue Befehl `$ESTOP` mit Sperre nötig (siehe Befehle).

Modi und Zustände

L1/R1 schalten zwischen Manuell und Autonom, X startet die Linienfolge, Kreis stoppt sie. Dreieck wirkt aus jedem Zustand und sperrt, bis X gedrückt wird.

Dreieck sperrt alles, nur X gibt wieder frei



Modi der App · 3 Betriebszustände + Not-Halt

Nach dem Lösen der Sperre steht der Roboter im zuletzt gewählten Modus, fährt aber nicht von selbst los.

Daten von der App an den Pico 2w (Eingänge)

Die App sendet Textzeilen `$NAME,args\n` auf den RX-Kanal, per Write Without Response. Für das manuelle Fahren reicht das vorhandene Protokoll. Modi, Not-Halt mit Sperre und Linienfolge brauchen neue Befehle in der Firmware.

Manuelles Fahren aus dem Controller

```
\begin{aligned}
x &= \text{linker Stick (links/rechts)} \cdot 100 \\
y &= (\text{R2} - \text{L2}) \cdot 100
\end{aligned}
```

- Solange R2, L2 oder der Stick betätigt sind: `$J,<x>,<y>` alle 50–100 ms senden, auch ohne Änderung.
- Alles losgelassen: einmal `$J,0,0`.
- Der Pico rechnet daraus links = y + x und rechts = y − x, multipliziert mit dem Höchsttempo. Kommt 300 ms nichts, hält er selbst an.

Vorhandene Befehle

Befehl	Auslöser in der App
`$J,x,y`	R2/L2 + linker Stick (Manuell)

Befehl	Auslöser in der App
`$STOP`	Kreis (Aus)
`$SPEED,n`	Steuerkreuz ↑/↓, Feinmodus (Quadrat → `$SPEED,30`)
`$FMT,JSON`	Einmal nach jedem Verbinden
`$INFO`	Einmal nach jedem Verbinden (Version prüfen)
`$PING`	Verbindungstest

Neue Befehle (Firmware-Erweiterung nötig)

Befehl	Auslöser	Wirkung im Pico 2 w	Antwort
`$ESTOP`	Dreieck	Aktiv bremsen (wie `$BRAKE`), Linienfolge beenden, **alle Fahrbefehle sperren**	`{"t":"estop","active":true}`
`$START`	X	Sperre aufheben; im Modus Autonom die Linienfolge starten	`{"t":"start","mode":"line"}`
`$MODE,MANUAL` / `$MODE,LINE`	L1 / R1	Modus setzen; beim Wechsel wird angehalten (ausrollen)	`{"t":"mode","value":"line"}`
`$LINEKP,<wert>`	Steuerkreuz ←/→ im Modus Autonom	Lenkstärke der Linienfolge	`{"t":"linekp","value":0.6}`
`$BIAS,<wert>`	Steuerkreuz ←/→ im Modus Manuell	Kurs-Trim für Geradeauslauf (in %)	`{"t":"bias","value":0.5}`
`$TRIMRESET`	Touchpad-Klick	Gelernten Motor-Trim löschen	`msg`

Befehl	Auslöser	Wirkung im Pico 2 w	Antwort
`$HB`	alle 500 ms, immer	Lebenszeichen: bleibt es 1,5 s aus, hält der Pico auch die Linienfolge an	–

Neue Felder in der status-Zeile

Feld	Werte	Bedeutung
`mode`	zusätzlich `line`	Linienfolge aktiv
`estop`	`true` / `false`	Not-Halt-Sperre aktiv
`gap`	`true` / `false`	Linienfolge in einer Lücke (keine Linie, fährt geradeaus)
`blocked`	`true` / `false`	Steht wegen Hindernis

Bildschirm, Sicherheit und offene Entscheidungen

Ansichten der App (Wechsel per Touchpad-Wischen)

- **Fahren:** siehe Figma
- **Autonom:** siehe Figma
- **Log:** Alle empfangenen Zeilen mit Zeit, Ereignisse farbig; Aufzeichnung als CSV teilen.

Sicherheitsregeln

- Not-Halt hat Vorrang vor allem und wirkt auch, wenn die App gerade im Hintergrund war: Dreieck wird sofort gesendet, ohne Warteschlange.
- Controller getrennt → App sendet sofort `$STOP` und blockiert Fahrbefehle, bis er wieder da ist.
- App geht in den Hintergrund → `$STOP`. Im Manuell-Modus hält der Pico sowieso nach 300 ms ohne `$J` an.
- BLE getrennt → der Pico hält selbst an (schon umgesetzt), die App verbindet automatisch neu und startet im Modus Manuell, nicht fahrend.
- Nach jedem Verbinden: `$FMT,JSON`, `$INFO`, Modus und Sperre aus der status-Zeile übernehmen (der Pico ist die Wahrheit).

Offene Entscheidungen

- [x] ~~Linker Stick zum Lenken und L2 für rückwärts – passt das so?~~
- [x] ~~Soll R2 im Modus Autonom das Tempo der Linienfolge regeln oder nur im Modus Manuell wirken?~~
- [x] ~~Gibt es weitere Modi außer Manuell und Autonom (z. B. Kalibrieren)? Davon hängt ab, ob L1/R1 nur umschalten oder durch eine Liste blättern.~~
- [x] ~~Soll Kreis im Modus Autonom auch die Linienfolge beenden oder nur pausieren?~~
- [x] ~~Heartbeat `$HB` einführen (Vorschlag: ja)?~~
- [x] ~~Welche Vorschläge für Touchpad, Quadrat, Steuerkreuz, Options und Share übernehmen?~~

Wenn diese Punkte entschieden sind, werden zuerst die neuen Befehle in der Firmware (Branch `app`) umgesetzt, dann die iOS-App.